

FX-aware Settlement Agent

An autonomous agent that settles cross-currency stablecoin payments at the best available on-chain rate on Arc.

Project Description

Cross-border and agentic commerce constantly convert between currencies. Converting at the naive spot rate leaves money on the table whenever a stablecoin (e.g. EURC) trades off-peg on-chain. The FX-aware Settlement Agent **prices the route before it pays**: it reads the live on-chain USDC/EURC rate on Arc, compares it to the real-world EUR/USD rate, computes the basis, and routes each payment the cheaper way — an on-chain swap to capture the dislocation, or a direct USDC transfer when there's no edge. It then settles in USDC via a Circle Developer-Controlled Wallet (server-side signing — no private keys handled by the app), attaches a memo for reconciliation, and charges a sub-cent per settlement fee (Nanopayments concept). It runs autonomously, per payment.

Track Track 4 — Best Agentic Economy Experience on Arc

Circle Account Email changepa@gmail.com

GitHub (open source, MIT) github.com/minimaker1/arc-settlement-agent

Demo video youtu.be/GoKS3b-9Cik

Demo application URL Local app — <http://localhost:8000> (run `python web.py`; setup in README). The video shows it live, and every settlement is verifiable on the Arc explorer.

Circle Products Used (on Arc)

- **USDC** — settlement rail, native on Arc; USDC-as-gas removes native-token funding.
- **Circle Developer-Controlled Wallets** — autonomous, server-side-signed settlement; no raw private keys in the app.
- **Nanopayments** — sub-cent per-settlement service fee.
- **StableFX (conceptual)** — multi-currency routing target; the route logic is built to plug in once access is granted.

Working MVP

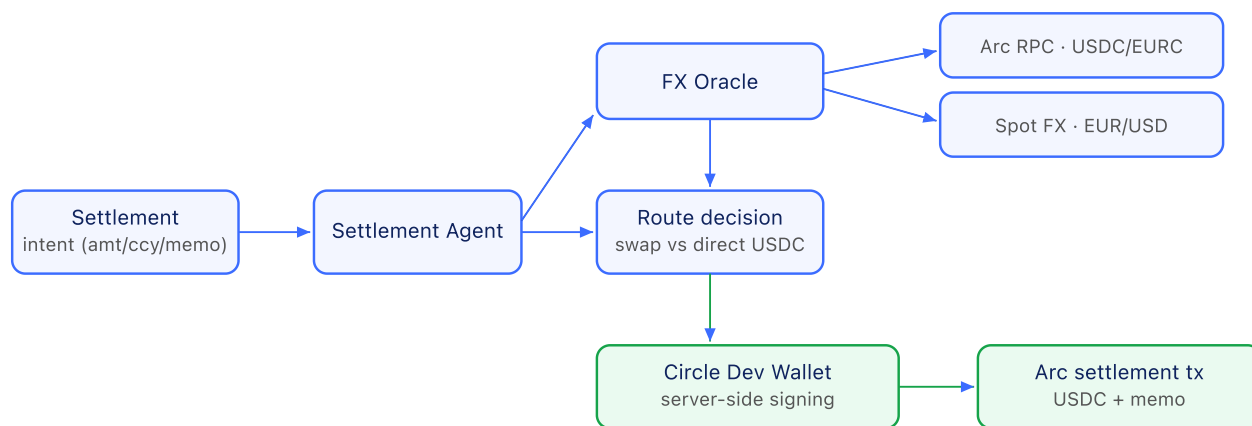
- **Backend:** `agent.py` (orchestration) + `fx_oracle.py` (on-chain basis vs spot) + `circle_wallet.py` (Circle Dev-Controlled Wallets) + `arc_client.py` (read-only Arc JSON-RPC).
- **Frontend:** `web.py` — one-page UI (`/present`) that plans a settlement and executes a real one on-chain. Run: `python web.py` → <http://localhost:8000>
- Runs live against Arc testnet (chainId 5042002, USDC 0x3600..., EURC 0x89B5...).

✅ **Real on-chain settlement (Arc testnet):** 1 USDC, FX-aware route, server-side signed by a Circle Developer-Controlled Wallet, memo attached.

tx [0xbef17f3513914f502012c81fcb4e7252464e6306b8f8a6e5238f9d302691234f](https://testnet.arcscan.app/tx/0xbef17f3513914f502012c81fcb4e7252464e6306b8f8a6e5238f9d302691234f)

testnet.arcscan.app/tx/0xbef17f35...91234f

Architecture



Documentation

Full setup, run instructions, architecture, and reproducible real-settlement demo are in the repository `README.md` (plus `circle_setup.py` for wallet provisioning and `demo_settle.py` for the on-chain demo): github.com/minimaker1/arc-settlement-agent

Circle Product Feedback

Why these products: USDC + Developer-Controlled Wallets let the agent settle autonomously with server-side signing — we never handle a private key, the exact trust model an agentic payment system needs. USDC-as-gas on Arc removed the native-token funding step. Nanopayments fits per-settlement pricing.

What worked well: Reading Arc over plain JSON-RPC worked first try against the documented USDC/EURC addresses. Wallet provisioning (entity secret → wallet set → ARC-TESTNET wallet) and the testnet faucet were smooth, and deterministic sub-second finality made settlement receipts trivial to verify — our 1-USDC tx confirmed on arcscan immediately.

What could be improved: (1) The `/developer/transactions/transfer` endpoint rejected the SDK-style nested `fee:{type:'level',config:{feeLevel}}`; the raw REST API needs a top-level `feeLevel`, but the 400 pointed at `gasPrice/gasLimit`, which was misleading. (2) Entity-secret registration is Console-only — there's no working REST endpoint (`/config/entity/entitySecret/register` → 404), which blocks fully-scripted setup. (3) DEX pool addresses (e.g. a Synthra USDC/EURC pool) aren't discoverable from the docs or the Uniswap registry, so wiring an on-chain price source needed UI hunting.

Recommendations: Document the raw-REST `feeLevel` shape with a copy-paste transfer example; expose a REST entity-secret registration path (or flag it Console-only in the quickstart); and publish a machine-readable testnet registry of DEX/pool addresses and StableFX corridors so oracle/routing builders can integrate programmatically.